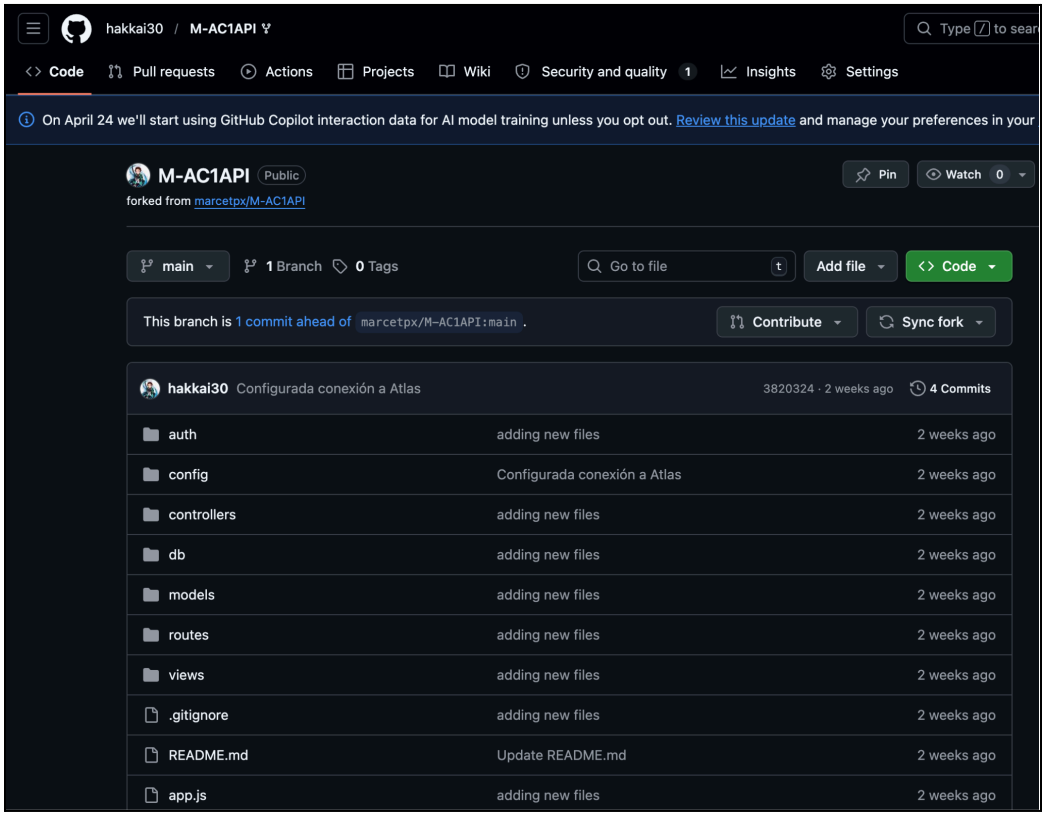
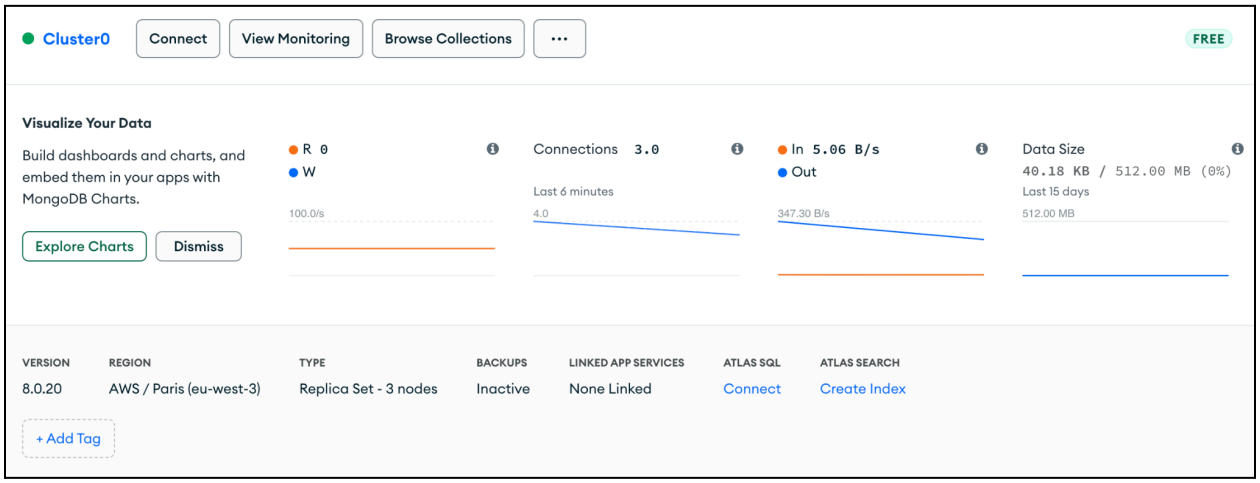
link render: https://m-ac1api.onrender.com/api

Tarea 1. Realiza un fork del siguiente repositorio a tu GitHub, y clona este a tu máquina local. https://github.com/marcetpx/M-AC1API/tree/main



Tarea 2. Sigue las instrucciones que encontrarás en el enlace siguiente para crear la base de datos en Atlas. https://www.mongodb.com/docs/atlas/getting-started/ Ten en cuenta que, para que nuestra aplicación pueda utilizar el clúster debes asegurarte de hacer las siguientes operaciones:

a. Crear un base de datos llamadas products. Las colecciones que contenga (el análogo a las tablas en el modelo relacional), las creará la propia aplicación.



b. Crear un nuevo usuario. Es recomendable que la contraseña asociada la genere automáticamente la UI de Atlas. Asegúrate de que el nuevo usuario creado dispone de todos los permisos sobre la base de datos.

```
_id: ObjectId('69c18687cd9413174fb7bd9b')
name : "robin"
password : "123"
apiKey : "gkmrqdnp2e83q2ydbd0fq00ruwhp02"
```

c. Añadir la IP desde la que vas a trabajar a la whitelist del clúster (en la UI de Atlas: Data Services/Security/Network Access). Puedes saber la IP pública desde la que te estas conectando con esta herramienta, por ejemplo:

https://www.whatismyip.com/es/.

Su IPv4 público es: 80.25.150.90

Su IPv6 público es:: Not Detected

-Finalmente, añade la URI de la connexión (la UI de Atlas: Data Services/Database/Connect) al fichero /config/config.js del proyecto.

```
mongodb+srv://robsgm30_db_user:<db_password>@cluster0.ghvylue.mongodb.net/?
appName=Cluster0
```

Tarea 3. Como puedes observar, el repositorio no contiene la carpeta de dependencias node_modules. Esta es una buena práctica en tiempo de desarrollo: ¿para qué subir los paquetes de dependencias, que son de largo los más pesados del proyecto, al repositorio (o a Clickedu...), si puede reconstruirse a partir del fichero package.json?

```
Node.js v22.14.0
[nodemon] app crashed - waiting for file changes before starting...
[nodemon] restarting due to changes...
[nodemon] starting `node app.js`
server is listening on port 5000
```

Wellcome to Products API

Register to get API key!

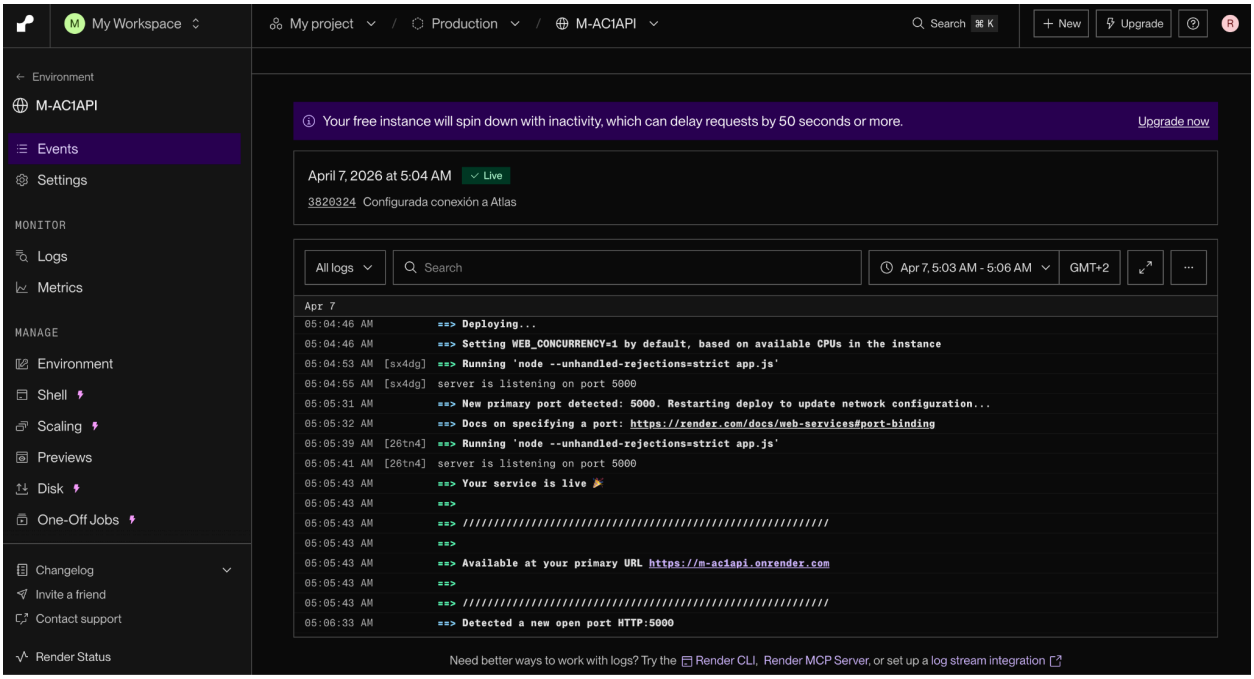
(to provide it, place it on header 'api-key' or as URL param 'apikey')

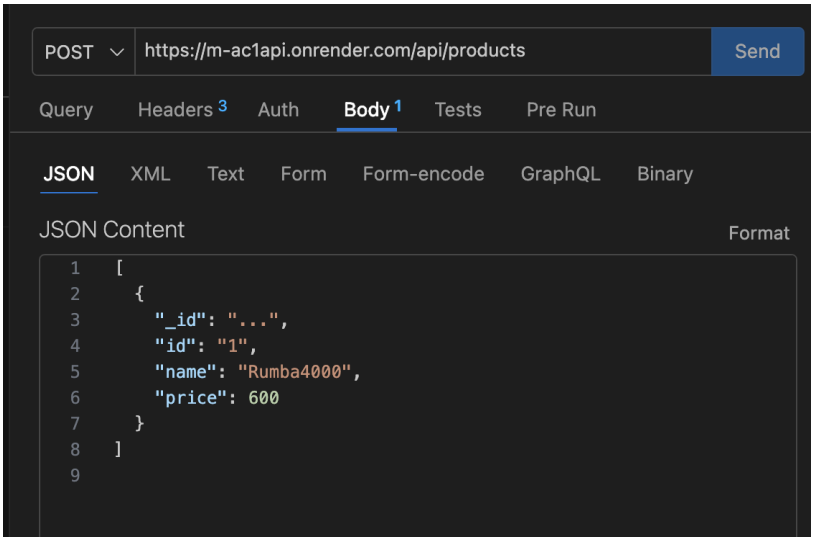
Operation	API endpoint
Register	POST /api/users/register
List all products	GET /api/products
List products by ID	GET /api/products/:productID
Create product	POST /api/products
Update product	PUT /api/products/:productID
Delete product	DELETE /api/products/:productID

-Despliegue del proyecto online (Render)

Tarea 4. A continuación, desplegaremos el proyecto online. Una buena solución para un aplicación NodeJS sería utilizar el hosting de Heroku, pero recientemente han retirado su free tier. Por ello, utilizaremos una alternativa parecida:

https://render.com/





Tarea 5. El hecho de que el web service se compile y despliegue automáticamente desde el repositorio de GitHub facilita enormemente el proceso de actualización del proyecto. Hagamos un cambio tonto para tener claro el proceso:

```
robin@Robins-MacBook-Air M-AC1API % npm install slugify

up to date, audited 178 packages in 606ms

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
Apr 7
04:49:03 PM  [jvvqh]     },
04:49:03 PM  [jvvqh]     stale: false,
04:49:03 PM  [jvvqh]     compatible: true,
04:49:03 PM  [jvvqh]     heartbeatFrequencyMS: 10000,
04:49:03 PM  [jvvqh]     localThresholdMS: 15,
04:49:03 PM  [jvvqh]     setName: 'atlas-9sovfp-shard-0',
04:49:03 PM  [jvvqh]     maxElectionId: null,
04:49:03 PM  [jvvqh]     maxSetVersion: null,
04:49:03 PM  [jvvqh]     commonWireVersion: 0,
04:49:03 PM  [jvvqh]     logicalSessionTimeoutMinutes: null
04:49:03 PM  [jvvqh]   },
04:49:03 PM  [jvvqh]   code: undefined,
04:49:03 PM  [jvvqh]   [Symbol(errorLabels)]: Set(0) {}
04:49:03 PM  [jvvqh] }
05:00:37 PM          ==> Deploying...
05:00:37 PM          ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
05:00:49 PM  [xdjhb] ==> Running 'node --unhandled-rejections=strict app.js'
05:00:51 PM  [xdjhb] server*is*listening*on*port*5000
05:00:58 PM          ==> Your service is live 🎉
05:00:59 PM          ==>
05:00:59 PM          ==> ///////////////////////////////////////////////////////////
05:00:59 PM          ==>
05:00:59 PM          ==> Available at your primary URL https://m-ac1api.onrender.com
05:00:59 PM          ==>
05:00:59 PM          ==> ///////////////////////////////////////////////////////////
```

Tarea 6. Habitualmente, el proceso de despliegue de una aplicación web habrá de venir precedido no solo de la instalación de las dependencias, sino de la compilación de ciertos ficheros. Estos son algunos ejemplos:

Wellcome to Products API

Register to get API key!

(to provide it, place it on header 'api-key' or as URL param 'apikey')

Operation	API endpoint
Register	POST /api/users/register
List all products	GET /api/products
List products by ID	GET /api/products/:productID
Create product	POST /api/products
Update product	PUT /api/products/:productID
Delete product	DELETE /api/products/:productID

-Ocultación de secretos:

Tarea 7. Lo más probable es que GitHub te haya avisado, mediante un correo, de que existe en tu repositorio una vulnerabilidad de seguridad. Concretamente, de que la URI de la conexión a la base de datos de MongoDB está expuesta, en texto plano, en el repositorio. Esto es gravísimo.

```
module.exports = {
  mongoURI: process.env.MONGOURI
};        marcetpx, 2 weeks ago • ad
```